LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE

Pertemuan 6 — State Management dengan GetX

Nama Mahasiswa	Bastian Yusuf Iskandar
NIM	3337240088
Mata Kuliah	Pemrograman Mobile
Pertemuan	Pertemuan 6
Topik	State Management dengan GetX
Studi Kasus Project	TokoKita — mengelola state keranjang belanja dengan GetX
Framework	Flutter (Dart) — GetX
Alokasi Waktu	170 menit (3 SKS)

A. Tujuan Praktikum

1. **Memahami kebutuhan state management** pada aplikasi Flutter berskala menengah-besar — terbukti dari permasalahan nyata TokoKita di Pertemuan 5, di mana jumlah item keranjang hilang begitu berpindah tab karena setiap halaman mengelola state-nya sendiri-sendiri. ✓
2. **Menggunakan konsep dasar GetX** — membuat class `CartController extends GetxController`, mendeklarasikan variabel reaktif `RxList` dengan ekstensi `.obs`, serta memantau perubahan data melalui widget `Obx`. ✓
3. **Menerapkan routing menggunakan GetX** — `Get.to()` untuk membuka halaman detail, `Get.back()` untuk kembali, dan `Get.off()` untuk berpindah halaman pasca-checkout tanpa dapat kembali ke halaman checkout. ✓
4. **Menerapkan dependency injection sederhana** — `Get.put(CartController())` pada awal aplikasi (`MainPage`) dan `Get.find<CartController>()` pada `HomePage`, `ProductDetailPage`, `CartPage`, dan `CheckoutPage`. ✓
5. **Membandingkan GetX dengan `setState`** secara konseptual (minimal 3 aspek) dan menganalisis kapan state lokal `setState` masih layak digunakan (status favorit `ProductCard`). ✓

B. Alat dan Bahan

B.1 Perangkat & Perangkat Lunak

- Laptop/PC dengan proyek **tokokita** beserta navigasi & bottom navigation hasil **Pertemuan 5**.
- Editor kode: Visual Studio Code / Android Studio.
- Android Emulator (atau perangkat fisik).

- Package **get (GetX)** yang telah ditambahkan pada `pubspec.yaml`.

B.2 Konfigurasi Proyek

```
name: tokokita
environment:
  sdk: ^3.13.2

dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8
  http: ^1.6.0
  get: ^4.7.3        # ← package GetX (Langkah 1)
```

Setelah dependensi ditambahkan, dijalankan perintah:

```
flutter pub get
```

B.3 Berkas yang Dibuat / Diubah

No	Berkas	Status	Peran pada Praktikum
1	`pubspec.yaml`	Diubah	Menambahkan dependency `get: ^4.7.3`
2	`lib/main.dart`	Diubah	`MaterialApp` → `GetMaterialApp` + `getPages`
3	`lib/controllers/cart_controller.dart`	**Baru**	`GetxController` berisi state keranjang
4	`lib/models/cart_item.dart`	**Baru**	Pembungkus `Product` + `quantity`
5	`lib/screens/main_page.dart`	Diubah	`Get.put(CartController())` + badge `Obx`
6	`lib/screens/home_page.dart`	Diubah	`Get.find()`, badge `Obx`, `Get.to()`
7	`lib/screens/product_detail_page.dart`	Diubah	`Get.find()`, `Get.back()`, `addToCart()`
8	`lib/screens/cart_page.dart`	Diubah	Tampilan nyata berbasis `Obx` + `ListView.builder`
9	`lib/screens/checkout_page.dart`	**Baru**	Ringkasan pesanan + `Get.off()` + `OrderSuccessPage`

C. Dasar Teori Singkat

Pada Pertemuan 5, data keranjang TokoKita masih dikirim antar halaman secara manual melalui `Navigator` dan belum tersimpan secara terpusat — jumlah item di keranjang akan hilang begitu berpindah tab, karena setiap halaman mengelola state-nya sendiri-sendiri. Kondisi ini adalah contoh nyata mengapa dibutuhkan pendekatan state management yang lebih terstruktur ketika data perlu dibagikan antar banyak widget yang tidak berada dalam satu hierarki langsung.

GetX adalah salah satu library state management populer di Flutter yang menawarkan **tiga pilar utama** dalam satu paket ringan:

Pilar	Fungsi	Penerapan di TokoKita
State Management	Memisahkan logika/data dari tampilan melalui `GetxController`; perubahan reaktif dibungkus tipe `Rx` dan dipantau widget `Obx`	`CartController`, `RxList<CartItem>`, `Obx`
Route Management	Navigasi tanpa `BuildContext`	`Get.to()`, `Get.back()`, `Get.off()`, `Get.offAll()`
Dependency Management	Mendaftarkan & mengambil instance controller dari memori	`Get.put()` dan `Get.find()`

C.1 Variabel Reaktif `Rx` dan Widget `Obx`

- `RxList<CartItem> items = <CartItem>[].obs;` — daftar yang secara otomatis mengirim notifikasi setiap kali isi/strukturnya berubah.
- `Obx(() { ... })` — widget yang **hanya** membangun ulang (*rebuild*) blok di dalamnya ketika variabel reaktif yang dibacanya berubah, tanpa me-rebuild seluruh halaman.
- Getter reaktif (`totalItem`, `totalHarga`) dihitung terpusat di controller sehingga semua halaman selalu menampilkan angka yang sama.

C.2 Perbandingan Navigasi GetX vs Navigator

GetX	Padanan Navigator	Sifat
`Get.to(page)`	`Navigator.push`	Menumpuk halaman baru ke atas stack
`Get.back()`	`Navigator.pop`	Kembali ke halaman sebelumnya
`Get.off(page)`	`Navigator.pushReplacement`	Mengganti halaman aktif (tak bisa kembali)
`Get.offAll(page)`	`Navigator.pushAndRemoveUntil`	Membersihkan seluruh tumpukan rute

C.3 Dependency Injection

- `Get.put(CartController())` → **mendaftarkan** satu instance controller ke memori aplikasi (sekali, saat awal aplikasi berjalan).
- `Get.find<CartController>()` → **mengambil** kembali instance tersebut dari mana saja, sehingga seluruh halaman berbagi satu sumber data yang sama (*single source of truth*).

D. Langkah Kerja

Langkah 1 — Instalasi & Konfigurasi GetX

1. Menambahkan package `get` pada `pubspec.yaml` lalu menjalankan `flutter pub get`.
2. Mengubah widget root aplikasi dari `MaterialApp` menjadi `GetMaterialApp` pada `lib/main.dart`:
   `dart return GetMaterialApp( title: namaToko, debugShowCheckedModeBanner: false, theme: ThemeData(...), initialRoute: '/', getPages: [ GetPage(name: '/', page: () => const MainPage()),`

`GetPage(name: '/detail', page: () => const ProductDetailPage()), GetPage(name: '/cart', page: () => const CartPage()), GetPage(name: '/checkout', page: () => const CheckoutPage()), ], );`

3. **Verifikasi:** aplikasi TokoKita tetap berjalan normal setelah perubahan tersebut, sekaligus mengaktifkan routing tanpa `BuildContext`, serta overlay snackbar dan dialog GetX di seluruh pohon widget.

Langkah 2 — CartController & Variabel Reaktif

1. Membuat berkas baru `lib/controllers/cart_controller.dart` berisi class berikut, serta model pendukung `lib/models/cart_item.dart` (membungkus objek `Product` + `quantity` + getter `subtotal`):
   ```dart
   class CartController extends GetxController {
   // Butir 1: variabel reaktif untuk menyimpan item keranjang
   final RxList items = [].obs;

   // Butir 3: getter reaktif totalItem & totalHarga
   int get totalItem => items.fold(0, (sum, item) => sum + item.quantity);
   double get totalHarga => items.fold(0.0, (sum, item) => sum + item.subtotal);
   ```
   `2. Membuat method mutasi data `addToCart(Product product, [int quantity = 1])` dan `removeFromCart(Product product)`. Jika produk sudah ada di keranjang, kuantitasnya ditambahkan dengan tetap memvalidasi batas stok (melebihi stok akan dipotong dan memunculkan `Get.snackbar`). 3. Getter reaktif `totalItem` dan `totalHarga` dihitung terpusat dengan `items.fold(...)`, sehingga selalu sinkron dengan isi keranjang. 4. Mendaftarkan controller ke memori aplikasi dengan **Dependency Injection** pada `MainPage` (`lib/screens/main_page.dart`):` dart
   ```
   final CartController _cartController = Get.put(CartController());
   ```

Langkah 3 — Obx & Reaktivitas Tampilan

1. Mengambil instance controller secara global dengan `Get.find<CartController>()` pada `HomePage`, `ProductDetailPage`, `CartPage`, dan `CheckoutPage` — tanpa mengoper data manual lewat parameter widget:
   `dart final CartController _cartController = Get.find<CartController>();`
2. Membungkus ikon keranjang (badge jumlah item) dengan `Obx` pada header `HomePage`, `AppBar` `ProductDetailPage`, dan `BottomNavigationBar` `MainPage`:
   `dart Obx(() { final total = _cartController.totalItem; return IconButton( icon: Badge( isLabelVisible: total > 0, label: Text('$total'), backgroundColor: const Color(0xFFE53935), child: const Icon(Icons.shopping_cart_outlined), ), onPressed: () => Get.to(() => const CartPage()), ); })`
3. Tombol **"Tambah ke Keranjang"** pada `ProductDetailPage` dipanggilkan langsung ke `_cartController.addToCart(product, _quantity)`.
   Hasil pengamatan: badge angka keranjang bertambah secara instan tanpa satu pun pemanggilan `setState()`.

Langkah 4 — Routing dengan GetX

1. **Navigasi maju (`Get.to`)** — menggantikan `Navigator.pushNamed(context, '/detail')` pada `HomePage`:
   `dart Get.to(() => ProductDetailPage(product: product));`

2. **Navigasi mundur (`Get.back`)** — menggantikan `Navigator.pop(context)` pada tombol panah `AppBar` `ProductDetailPage` :
   `dart Get.back();`
   Berjalan mulus tanpa objek `BuildContext` .
3. **Navigasi ganti tumpukan (`Get.off`)** — mensimulasikan perpindahan halaman setelah proses checkout:
   - Tombol **Checkout** pada `CartPage` → `Get.off(() => const CheckoutPage());`
   - Tombol **Bayar Sekarang** pada `CheckoutPage` → `cartController.clearCart(); Get.off(() => const OrderSuccessPage());`
   - Pengguna **tidak dapat** kembali ke halaman checkout (mencegah duplikasi pembayaran).
   - Tambahan: tombol "*Kembali ke Beranda*" pada `OrderSuccessPage` memakai `Get.offAll(() => const MainPage())` untuk membersihkan seluruh tumpukan rute.

Langkah 5 — Menampilkan Halaman Keranjang dengan CartController

1. Mengganti *placeholder* Keranjang Pertemuan 5 dengan tampilan nyata berbasis `Obx` + `ListView.builder` yang membaca `cartController.items` .
2. Menambahkan tombol hapus pada setiap item yang memanggil `cartController.removeFromCart(product)` ; tampilan ter-update otomatis. Setiap baris memuat thumbnail kategori, nama produk, label kategori, batas stok, harga satuan, pengatur kuantitas (−/+), dan subtotal per baris.
3. Menampilkan `totalHarga` di bagian bawah halaman (`bottomNavigationBar` dibungkus `Obx`):
   `dart bottomNavigationBar: Obx(() { // menampilkan cartController.totalHarga (format rupiah) // + tombol Checkout -> Get.off(() => const CheckoutPage()); }),`
   Nilai total selalu konsisten dengan isi keranjang pada setiap penambahan, pengurangan, perubahan kuantitas, maupun penghapusan item.

Kondisi awal (keranjang kosong): `CartPage` menampilkan ilustrasi ikon keranjang, teks "*Keranjang Belanja Masih Kosong*", dan tombol ajakan "*Mulai Belanja Sekarang*" — tampilan ini otomatis berganti menjadi daftar item begitu `items` tidak kosong.

E. Hasil Pengamatan

Catat hasil pengujian penggunaan GetX pada tabel berikut:

No.	Fitur GetX yang Diuji	Hasil/Perilaku yang Diamati	Screenshot (Ya/Tidak)
1	**Badge keranjang reaktif dengan Obx**	Ketika produk ditambahkan dari `ProductDetailPage` , badge angka merah pada header `HomePage` , `AppBar` `ProductDetailPage` , dan `BottomNavigationBar` `MainPage` langsung diperbarui *real-time* tanpa rebuild seluruh halaman dan tanpa `setState` .	Ya
2	**`addToCart` & `removeFromCart`**	`addToCart` menambahkan produk baru ke `RxList<CartItem>` atau menaikkan kuantitas produk yang sudah ada (dengan batas stok + snackbar peringatan). `removeFromCart` menghapus item terpilih dan daftar keranjang langsung menyusut otomatis.	Ya

No.	Fitur GetX yang Diuji	Hasil/Perilaku yang Diamati	Screenshot (Ya/Tidak)
3	**`Get.to()` & `Get.back()` ke ProductDetailPage**	Menekan kartu produk di `HomePage` membuka detail via `Get.to(() => ProductDetailPage(product: product))` tanpa `BuildContext`; tombol panah `AppBar` memanggil `Get.back()` dan kembali mulus.	Ya
4	**`Get.find()` lintas halaman (Home, Detail, Keranjang)**	Controller yang didaftarkan sekali di `MainPage` lewat `Get.put(CartController())` dapat diakses konsisten di `HomePage`, `ProductDetailPage`, `CartPage`, dan `CheckoutPage` lewat `Get.find<CartController>()` — seluruh halaman berbagi satu *single source of truth*.	Ya
5	**Konsistensi `totalHarga` pada halaman Keranjang**	`totalHarga` dari getter reaktif selalu sinkron pada setiap perubahan: item ditambah, kuantitas naik/turun, item dihapus, maupun keranjang dikosongkan.	Ya

> **Catatan:** kolom *Screenshot* diisi setelah Anda menyisipkan tangkapan layar hasil uji masing-masing pada saat penyusunan laporan PDF.

F. Tugas Mandiri

1. Tambahkan fitur ubah jumlah (quantity) per item pada halaman Keranjang TokoKita, dengan `totalHarga` yang otomatis menyesuaikan menggunakan Obx.

Ditambahkan dua method pada `CartController`:

```
void increaseQuantity(Product product) {
  final existingIndex = items.indexWhere((item) => item.product.id == product.id);
  if (existingIndex >= 0) {
    final item = items[existingIndex];
    if (item.quantity < product.stock) {
      item.quantity++;
      items.refresh(); // memicu notifikasi perubahan pada Obx
    } else {
      Get.snackbar('Stok Terbatas',
          'Kuantitas maksimal produk "${product.name}" adalah ${product.stock} unit.',
          snackPosition: SnackPosition.BOTTOM,
          backgroundColor: const Color(0xFFEF4444), colorText: Colors.white);
    }
  }
}

void decreaseQuantity(Product product) {
  final existingIndex = items.indexWhere((item) => item.product.id == product.id);
  if (existingIndex >= 0) {
    final item = items[existingIndex];
    if (item.quantity > 1) {
      item.quantity--;
      items.refresh();
    } else {
      removeFromCart(product); // kuantitas < 1 -> item dihapus
    }
  }
}
```

Pada `CartPage`, setiap baris memiliki kontrol angka (tombol −, teks `${item.quantity}`, tombol +) yang dibungkus `Obx`, sehingga `item.subtotal` dan `cartController.totalHarga` dihitung ulang seketika tanpa jeda.

2. Tambahkan tombol 'Kosongkan Keranjang' pada CartController beserta konfirmasi sebelum dijalankan.

Method `confirmClearCart` memakai dialog bawaan GetX `Get.defaultDialog`:

```
void clearCart() => items.clear();

void confirmClearCart(BuildContext context) {
  if (items.isEmpty) return;

  Get.defaultDialog(
    title: 'Kosongkan Keranjang?',
    titleStyle: const TextStyle(fontSize: 18, fontWeight: FontWeight.w700),
    middleText: 'Apakah Anda yakin ingin menghapus semua barang dari keranjang belanja?',
    textConfirm: 'Ya, Kosongkan',
    textCancel: 'Batal',
    confirmTextColor: Colors.white,
    buttonColor: const Color(0xFFDC2626),
    radius: 12,
    onConfirm: () {
      clearCart();
      Get.back(); // menutup dialog
      Get.snackbar('Keranjang Dikosongkan',
          'Semua item berhasil dihapus dari keranjang belanja.',
          snackPosition: SnackPosition.BOTTOM,
          backgroundColor: const Color(0xFF1976D2), colorText: Colors.white);
    },
  );
}
```

Tombol pemicu diletakkan di `AppBar` kanan atas `CartPage` (ikon `Icons.delete_sweep_outlined`, warna merah), dan hanya muncul jika keranjang memiliki item (dibungkus `Obx`).

3. Buat halaman Checkout sederhana yang menampilkan ringkasan pesanan dari CartController, dan gunakan `Get.off()` untuk berpindah ke halaman tersebut dari halaman Keranjang.

Berkas baru `lib/screens/checkout_page.dart` berisi:
1. **Navigasi dari Keranjang** — tombol Checkout memanggil:
`dart Get.off(() => const CheckoutPage());`
`CartPage` tergantikan dalam tumpukan rute sehingga pengguna tidak dapat kembali ke keranjang yang sedang diproses.
2. **Ringkasan pesanan** — kartu alamat pengiriman, daftar item dari `cartController.items` (`ListView.separated`), dan rincian pembayaran (Total Harga Produk, Biaya Pengiriman *Gratis Promo*, Total Tagihan Akhir) yang totalnya dibungkus `Obx(() => Text(formatRupiah(cartController.totalHarga), ...))`.
3. **Simulasi selesai pembayaran** — `OrderSuccessPage` (dalam berkas yang sama):
`dart cartController.clearCart(); Get.off(() => const OrderSuccessPage());`
Mencegah pengguna kembali ke checkout (*double charging*), lalu dari halaman sukses tombol "*Kembali ke Beranda*" memakai `Get.offAll(() => const MainPage())`.

4. Tabel perbandingan antara pendekatan `setState` (Pertemuan 3, status favorit) dan GetX (Pertemuan 6, keranjang) mencakup minimal tiga aspek.

Aspek	`setState` (Lokal — Pertemuan 3)	`GetX` (Global/Terpusat — Pertemuan 6)
1. Lingkup state (*scope*)	State lokal (*ephemeral*) — hanya hidup di dalam satu `StatefulWidget`; berbagi data ke cabang pohon lain butuh *lifting state up* yang rumit.	State global (*shared*) — disimpan di `GetxController` di luar pohon widget; dapat diakses & dimutasi dari halaman mana saja via `Get.find()`.
2. Efisiensi & granularitas rebuild	`setState()` me-rebuild **seluruh** `build method` StatefulWidget, termasuk widget turunan yang datanya tidak berubah.	Hanya widget yang dibungkus `Obx` dan membaca variabel `Rx` terkait yang di-rebuild; UI di sekitarnya tidak ikut rebuild.
3. Pemisahan logika (*separation of concerns*)	Logika manipulasi data & kalkulasi bercampur dengan definisi tampilan di dalam class `State`.	Logika bisnis terisolasi rapi di `GetxController`; UI fokus pada rendering tampilan visual saja.
4. Ketergantungan `BuildContext`	Hampir semua operasi butuh konteks (`Navigator.of(context)`, `ScaffoldMessenger.of(context)`).	Bebas `BuildContext` — navigasi (`Get.to/back/off`), dialog (`Get.defaultDialog`), dan snackbar (`Get.snackbar`) bisa dijalankan langsung dari controller.

5. Mengapa data keranjang TokoKita lebih tepat dikelola dengan GetX, sementara status favorit pada ProductCard masih bisa menggunakan setState?

a. Keranjang belanja → GetX, karena karakteristik datanya bersifat **global dan lintas layar**: badge jumlah barang harus sinkron secara bersamaan di header `HomePage`, `BottomNavigationBar` `MainPage`, `AppBar` `ProductDetailPage`, tombol tambah item, daftar & total di `CartPage`, serta rincian pembayaran di `CheckoutPage`. Bila memakai `setState`, pengembang harus melakukan *lifting state up* ke widget paling atas lalu mengoper callback menembus banyak lapis widget (*prop drilling*) — kode menjadi rapuh, sulit dirawat, dan memicu *unnecessary rebuild*. Dengan GetX, `CartController` menjadi *single source of truth*: setiap perubahan (tambah, ubah kuantitas, hapus) hanya menyiarkan pembaruan ke widget pembungkus `Obx`.

b. Status favorit `ProductCard` → `setState`, karena termasuk **state lokal (*ephemeral*)**: interaksi pada satu kartu produk hanya berdampak pada kartu itu (ikon hati berubah warna), tidak memengaruhi header, badge keranjang, maupun halaman lain. `setState` pada `StatefulWidget` sudah sederhana, ringan, dan tidak memerlukan overhead controller terpisah.

Kesimpulan analisis: penentuan state management tidak didasarkan pada jenis halamannya, melainkan pada **jangkauan state (scope)**, **kebutuhan berbagi data antar halaman**, dan **siklus hidup data**.

G. Kesimpulan

1. Praktikum Pertemuan 6 berhasil menerapkan seluruh pilar GetX pada project TokoKita: **state management** (`CartController` + `RxList` + `Obx`), **route management** (`Get.to`, `Get.back`, `Get.off`, `Get.offAll`), dan **dependency management** (`Get.put` + `Get.find`).
2. Masalah *state* Pertemuan 5 (jumlah keranjang hilang saat berpindah tab) teratasi karena seluruh halaman kini membaca satu sumber data tunggal, sehingga badge, daftar item, dan `totalHarga` selalu sinkron

secara *real-time* tanpa `setState`.

3. Pembaruan tampilan dengan `Obx` bersifat **granular** — hanya bagian UI yang membaca variabel reaktif yang di-rebuild, sehingga lebih efisien dibanding `setState` yang me-rebuild seluruh widget.
4. GetX memisahkan logika bisnis dari tampilan dan menghilangkan ketergantungan `BuildContext` pada navigasi/dialog, menghasilkan struktur kode yang lebih rapi dan mudah dirawat.
5. State lokal sederhana seperti status favorit tetap lebih tepat memakai `setState`, sementara data global lintas halaman seperti keranjang belanja tepat dikelola dengan GetX.

Catatan pengumpulan (sesuai lembar praktikum):

- Laporan dikumpulkan dalam format **PDF**.
- Format nama file: `Praktikum6_NIM_NamaMahasiswa.pdf` → `Praktikum6_3337240088_BastianYusufIskandar.pdf`
- Dikumpulkan paling lambat sebelum pertemuan berikutnya dimulai.